CSE 2217/CSI 227: Data Structure and Algorithms-II

Asymptotic notation

Asymptotic Performance

- In this course, we care most about *asymptotic performance*
  - How does the algorithm behave as the problem size gets very large?
    - Running time
    - Memory/storage requirements
    - Bandwidth/power requirements/logic gates/etc.

Asymptotic Notation

- By now you should have an intuitive feel for asymptotic (big-O) notation:
  - *What does O(n) running time mean? $O(n^2)$?*

- Our first task is to define this notation more formally and completely

Analysis of Algorithms

- Analysis is performed with respect to a computational model
- We will usually use a generic uniprocessor random-access machine (RAM)
  - All memory equally expensive to access
  - No concurrent operations
  - All reasonable instructions take unit time
    - Except, of course, function calls

Input Size

- Time and space complexity
  - This is generally a function of the input size
    - E.g., sorting, multiplication
  - How we characterize input size depends:
    - Sorting: number of input items
    - Multiplication: total number of bits
    - Graph algorithms: number of nodes & edges
    - Etc

Running Time

- Number of primitive steps that are executed
  - Except for time of executing a function call most statements roughly require the same amount of time
    - y = m * x + b
    - c = 5 / 9 * (t - 32)
    - z = f(x) + g(y)

Analysis

- Worst case
  - Provides an upper bound on running time
  - An absolute guarantee
- Average case
  - Provides the expected running time
  - Very useful, but treat with care: what is "average"?
    - Random (equally likely) inputs
    - Real-life inputs

An Example: Insertion Sort

```
InsertionSort(A) {
  for j <- 2 to Length[A] {
      key <- A[j]
   //insert A[j] into a sorted sequence A[1…j-1]
       i <- j - 1;
   while (i > 0) and (A[i] > key) {
      A[i+1] <- A[i]
      i <- i - 1
   }
   A[i+1] <- key
  }
}
```

An Example: Insertion Sort (Cont..)

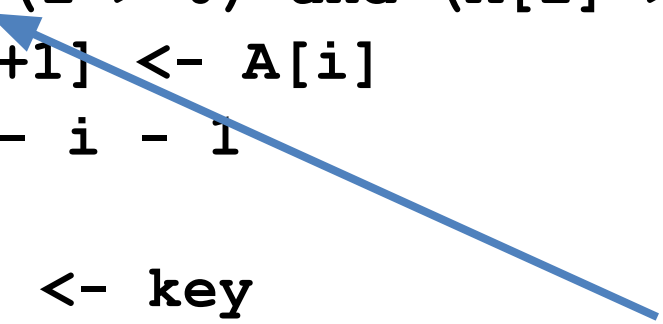
Problem: Sort the sequence {3, 7, 4, 9, 5, 2, 6, 1}

3 7 4 9 5 2 6 1
3 7 4 9 5 2 6 1
3 **7** 4 9 5 2 6 1
3 **4** 7 9 5 2 6 1
3 4 7 **9** 5 2 6 1
3 4 **5** 7 9 2 6 1
2 3 4 5 7 9 6 1
2 3 4 5 **6** 7 9 1
1 2 3 4 5 6 7 9

```
InsertionSort(A) {
   for j <- 2 to Length[A] {
        key <- A[j]
    //insert A[j] into a sorted A[1…j-1]
      i <- j - 1;
    while (i > 0) and (A[i] > key) {
        A[i+1] <- A[i]
        i <- i - 1
    }
    A[i+1] <- key
   }
}
```

Insertion Sort

```
InsertionSort(A) {
  for j <- 2 to Length[A] {
      key <- A[j]
   //insert A[j] into a sorted sequence A[1…j-1]
       i <- j - 1;
   while (i > 0) and (A[i] > key) {
      A[i+1] <- A[i]
      i <- i - 1
   }
   A[i+1] <- key
  }
}
```



How many times will this loop execute?

Insertion Sort

Statement	Cost	Times
`InsertionSort(A) {`		
`  for j <- 2 to Length[A] {`	c_1	n
`  key <- A[j]`	c_2	n-1
`  //insert A[j] into a sorted sequence A[1…j-1]`		
`  i <- j - 1;`	c_3	n-1
`  while (i > 0) and (A[i] > key) {`	c_4	$\sum_{j=2}^{n} t_j$
`  A[i+1] <- A[i]`	c_5	$\sum_{j=2}^{n} (tj - 1)$
`  i <- i - 1`	c_6	$\sum_{j=2}^{n} (tj - 1)$
`  }`		
`  A[i+1] <- key`	c_7	n-1
`  }`		
`}`		

t_j is number of while expression evaluations for the j^{th} for loop iteration

Analyzing Insertion Sort

- $T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum_{j=2}^{n} t_j + c_5 \sum_{j=2}^{n}(tj - 1) + c_6 \sum_{j=2}^{n}(tj - 1) + c_7(n-1)$

- What can T be?
  - Best case -- inner loop body never executed
    - $t_j = 1$ ➔ T(n) is a linear function
  - Worst case -- inner loop body executed for all previous elements
    - $t_j = j$ ➔ T(n) is a quadratic function

Analysis

- Simplifications
  - Ignore actual and abstract statement costs
  - *Order of growth* is the interesting measure:
    - Highest-order term is what counts
      - Remember, we are doing asymptotic analysis
      - As the input size grows larger it is the high order term that dominates

Rate of Growth $\equiv$ Asymptotic Analysis

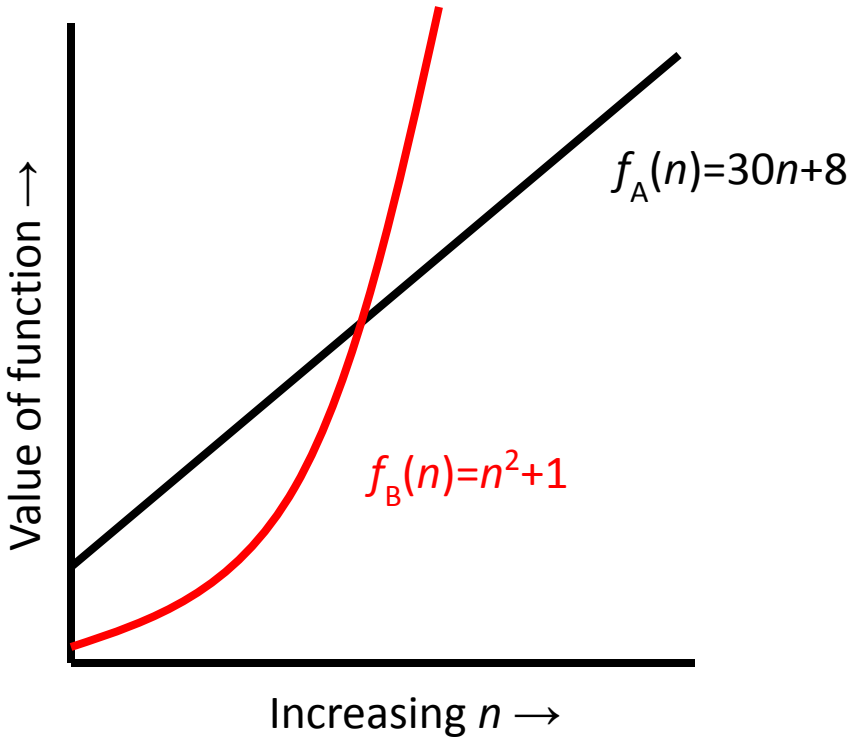
- Using *rate of growth* as a measure to compare different functions implies comparing them **asymptotically**.
- If $f(x)$ is *faster growing* than $g(x)$, then $f(x)$ always eventually becomes larger than $g(x)$ **in the limit** (for large enough values of x).

Example

- Suppose you are designing a web site to process user data (*e.g.*, financial records).
- Suppose program A takes $\boldsymbol{f_A(n)=30n+8}$ microseconds to process any n records, while program B takes $\boldsymbol{f_B(n)=n^2+1}$ microseconds to process the n records.
- Which program would you choose, knowing you'll want to support millions of users?

Visualizing Orders of Growth

- On a graph, as you go to the right, a faster growing function eventually becomes larger...



Asymptotic Complexity

- Running time of an algorithm as a function of input size *n* **for large *n***.
- Expressed using only the **highest-order term** in the expression for the exact running time.
  - Instead of exact running time, say $\Theta(n^2)$.
- Describes behavior of function in the limit.
- Written using ***Asymptotic Notation.***

Asymptotic Notation

- **Θ, *O*, Ω, *o*, ω**
- Defined for functions over the natural numbers.
  - **Ex:** $f(n) = \Theta(n^2)$.
  - Describes how $f(n)$ grows in comparison to n^2.
- Define a ***set*** of functions; in practice used to compare two function sizes.
- The notations describe different rate-of-growth relations between the defining function and the defined set of functions.

Upper Bound Notation

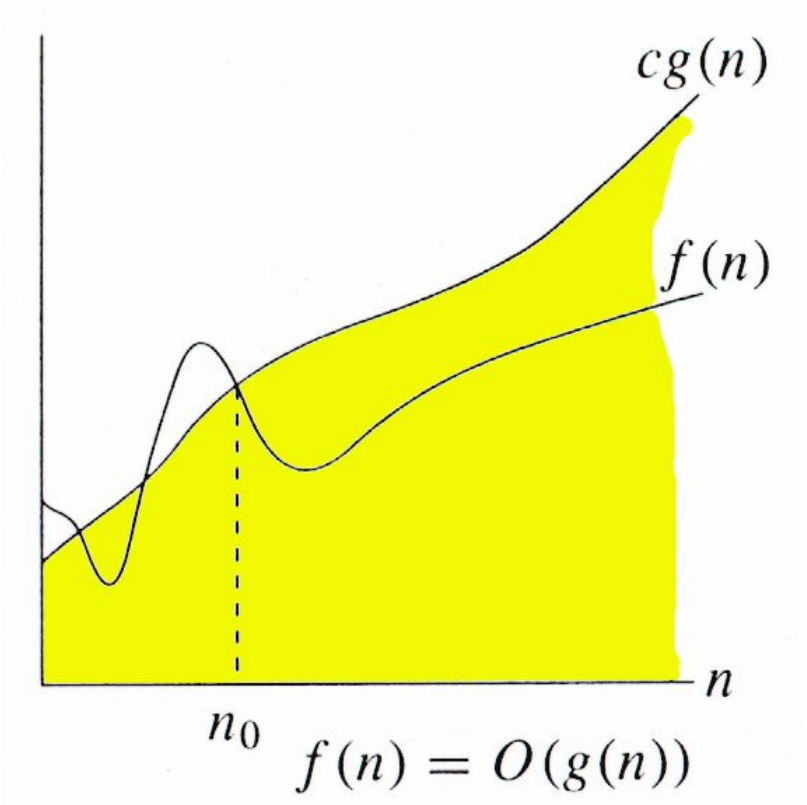
- We say InsertionSort's run time is $O(n^2)$
  - Properly we should say run time is *in* $O(n^2)$
  - Read O as "Big-O" (you'll also hear it as "order")
- In general a function
  - f(n) is O(g(n)) if there exist positive constants *c* and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$
- Formally
  - $O(g(n)) = \{ f(n): \exists$ positive constants *c* and n_0 such that $f(n) \leq c \cdot g(n) \ \forall \ n \geq n_0$

O-notation

For function $g(n)$, we define $O(g(n))$, big-O of n, as the set:

$O(g(n)) = \{f(n)$:
$\exists$ positive constants c and n_0,
such that $\forall n \geq n_0$,
we have $0 \leq f(n) \leq cg(n)\}$

Intuitively: Set of all functions whose *rate of growth* is the same as or lower than that of $g(n)$.



$g(n)$ is an *asymptotic upper bound* for $f(n)$.

Insertion Sort is $O(n^2)$

- Proof
  - The run-time is $an^2 + bn + c$
    - If any of a, b, and c are less than 0, replace the constant with its absolute value
  - $an^2 + bn + c \leq (a + b + c)n^2 + (a + b + c)n + (a + b + c)$

    $\leq 3(a + b + c)n^2$ for $n \geq 1$

    Let $c' = 3(a + b + c)$ and let $n_0 = 1$. Then

    $an^2 + bn + c \leq c' n^2$ for $n \geq 1$

    Thus $an^2 + bn + c = O(n^2)$.

Big-O Notation

- We say $f_A(n)=30n+8$ is *order n*, or O(*n*). It is, **at most**, roughly *proportional* to *n*.
- $f_B(n)=n^2+1$ is *order* n^2, or O(n^2). It is, **at most**, roughly proportional to n^2.
- In general, an O(n^2) algorithm will be <u>slower</u> than O*(n)* algorithm.
- **Warning**: an O(n^2) function will grow <u>faster</u> than an O*(n)* function.

More Examples …

- We say that $n^4 + 100n^2 + 10n + 50$ is of the order of n^4 or $O(n^4)$
- We say that $10n^3 + 2n^2$ is $O(n^3)$
- We say that $n^3 - n^2$ is $O(n^3)$

Even though it is correct to say "7n - 3 is $O(n^2)$", a better statement is "7n - 3 is O(n)", that is, one should make the approximation as tight as possible